

Project Synopsis – Smart Ride-Sharing Platform

DEPARTMENT OF COMPUTER APPLICATION

ACADEMIC SESSION: MARCH 2026 TO JUNE 2026

IV SEMESTER MCA

PROJECT SYNOPSIS

Title of the Project: **Smart Ride-Sharing Platform with Proactive AI Safety & Verified Communities**

Submitted by:

Name: Nidhi Rai

Register Number: 24P01095

Section: B

Under the Guidance of:

Guide Name: _____

1. Abstract

Urban transportation systems suffer from inefficient ride allocation, delayed customer support, and a fundamental lack of intelligent, proactive user interaction. Current market-leading platforms like Uber and Ola rely on manual support teams and rule-based chatbots that fail on complex queries, causing longer response times, unclear fares, booking difficulties, and critical safety concerns during rides.

The proposed Smart Ride-Sharing Platform addresses these gaps by integrating seven AI-powered modules: (1) a Multi-Turn Intent-Aware NLP Chatbot built on a TensorFlow/Keras neural network; (2) Multilingual Voice Booking (Hindi and English) via the Web Speech API; (3) a Proactive GPS Anomaly Detection Safety Engine — the "Silent Operator" — using Scikit-learn's Isolation Forest; (4) Domain-Verified Trust Circles with Reputation Scoring and women-only driver matching; (5) Carbon-Aware EV Routing with Gamified Green Rewards; (6) a Unified Safety & Operations Dashboard; and (7) an AI-Driven Ride Risk Score — a pre-trip passenger safety transparency feature absent from all commercial platforms.

This web-based system uses NLP, speech recognition, Scikit-learn Isolation Forest anomaly detection, and route-matching algorithms to automate customer support, enhance trust and safety, reduce manual intervention by 70%, and demonstrate practical ML applications in transportation.

2. Introduction

Ride-sharing platforms have transformed urban mobility but face persistent challenges including slow customer support, inefficient ride matching, safety vulnerabilities, and limited conversational interfaces. Traditional apps like Uber and Ola provide basic booking and GPS tracking yet depend heavily on manual support teams for query resolution, leading to delays and suboptimal user experiences.

This project introduces a Smart Ride-Sharing Platform enhanced with neural network-powered AI. Key innovations include:

- Multi-Turn Intent-Aware NLP Chatbot (TensorFlow/Keras LSTM) with session context memory for multi-turn ride booking conversations.
- Multilingual Voice-Based Booking (English & Hindi) using the browser-native Web Speech API — directly relevant to the Indian commuter demographic.

- Proactive "Silent Operator" GPS Anomaly Detection Engine using Scikit-learn's Isolation Forest, trained on the NYC Taxi Dataset, running as a Celery background task every 30 seconds.
- Domain-Verified Trust Circles with In-Community Reputation Scoring — beyond simple star ratings, trust compounds over shared ride history.
- Carbon-Aware EV Routing with Gamified Green Rewards (Green Points system with fare discounts).
- Unified Safety & Operations Dashboard combining live map, anomaly alerts, and community stats via Socket.io.
- AI-Driven Ride Risk Score computed before each trip — driver anomaly history, rating, time-of-day, and route zone risk — displayed as a colour-coded badge.

3. Problem Statement

What problem the project is solving:

The project addresses key challenges in urban ride-sharing systems: inefficient ride allocation and matching, delayed customer support for queries like fare estimation and cancellations, safety risks during rides (e.g., route deviations or unusual stops), lack of trust between strangers, and absence of eco-friendly or pooled ride options. It also tackles poor user interaction through non-intuitive menus and purely reactive emergency features that require the passenger to manually trigger an SOS.

A critical gap exists: no commercial ride-sharing platform currently provides a pre-trip AI-computed risk score for drivers, nor a proactive, always-on background safety monitor that intervenes automatically without user action when anomalous GPS behaviour is detected.

Why the current solutions are insufficient:

Existing platforms like Uber and Ola rely on manual customer support and basic rule-based chatbots, leading to long response times, unclear fare estimates, and frustrating navigation. They offer only reactive SOS buttons without proactive anomaly detection, no domain-verified community matching or women-only modes to build trust, no multilingual voice interfaces for the Indian market, and minimal support for green vehicles or intelligent pooling — resulting in scalability issues, safety gaps, higher costs, and suboptimal user experiences.

4. Objectives of the Project

- To develop a Multi-Turn Intent-Aware NLP Chatbot using a TensorFlow/Keras LSTM model achieving $\geq 85\%$ accuracy in intent classification for instant natural language query handling (booking, cancellations, fares).
- To implement a Proactive "Silent Operator" GPS Anomaly Detection Engine using Scikit-learn's Isolation Forest algorithm, trained on the NYC Taxi Dataset, for real-time background ride safety monitoring without user intervention.
- To create Domain-Verified Trust Circles with Reputation Scoring — verified by institutional email domains — and enable a hard-filter women-only driver matching mode to enhance user trust and safety.
- To integrate Multilingual Voice-Based Ride Booking (English and Hindi) with the browser-native Web Speech API and spaCy NLP entity extraction for hands-free interaction relevant to the Indian market.
- To enable Carbon-Aware EV Routing with a Gamified Green Rewards System, intelligent ride pooling via a proximity-based route-matching algorithm, targeting 50% cost reduction and urban sustainability.
- To introduce an AI-Driven Ride Risk Score (pre-trip composite safety rating from driver anomaly history, ratings, time-of-day, and route zone risk) — a feature absent in all existing commercial platforms.
- To design a scalable web-based platform with a Unified Safety & Operations Dashboard that reduces manual support dependency by 70% and demonstrates practical ML integration in transportation.

5. Literature Survey / Existing System

System / Product	AI Chatbot	Safety Features	Trust / Community	Voice Booking	Pooling / Green	Key Limitations
Uber	Basic rule-based	Reactive SOS only	Rating system	No	Limited (suspended)	Surge pricing, no proactive safety
Ola	Simple FAQ bot	SOS button	Driver ratings	No	Basic carpool	Frequent cancellations, poor support
Lyft	Rule-based chat	Emergency button	Ratings + women riders	No	Shared rides	No India presence, no anomaly detection
Bolt	Basic support	SOS	Ratings	No	Limited pooling	Poor NLP, manual support delays
Rapido	None	Basic SOS	Ratings	No	Bike pooling	No AI interaction, safety concerns
BluSmart	None	App tracking	Verified EVs	No	EV-only	No chatbot, limited cities
inDrive	None	Bid-based	User choice	No	No pooling	Trust issues, no safety AI
Yandex Go	Basic bot	SOS	Ratings	No	Limited	Region-specific, poor NLP
Grab	FAQ chatbot	SOS	Ratings	No	Pooling	SE Asia focus, reactive safety only
GoJek	Simple bot	Emergency	Ratings	No	Shared rides	Heavy manual support, no neural AI

Limitations of current systems:

All surveyed platforms share four core weaknesses: (1) rule-based bots that fail on complex or multi-turn queries; (2) purely reactive safety with no proactive GPS anomaly detection; (3) no domain-verified communities or women-only matching; and (4) no multilingual voice interfaces optimised for the Indian market.

Research Gap Identified:

No existing platform combines a proactive AI safety monitor, a pre-trip risk score, domain-verified communities, and multilingual voice booking in a unified system. This project specifically targets that gap.

6. Proposed System

The proposed Smart Ride-Sharing Platform is a web-based system integrating neural network-powered AI to revolutionize urban transportation by automating customer support, enhancing safety proactively, and optimizing ride efficiency.

Key Features & Solutions:

- **Multi-Turn Intent-Aware NLP Chatbot:** A TensorFlow/Keras LSTM model trained on a custom intent dataset (7+ classes) maintains session context across conversation turns and directly triggers backend booking actions when intent is confirmed.
- **Multilingual Voice-Based Booking (Hindi & English):** Browser-native Web Speech API processes voice commands; spaCy extracts pickup, dropoff, and ride-type entities without any paid third-party API.
- **Proactive "Silent Operator" GPS Anomaly Detection Engine:** Scikit-learn Isolation Forest ($n_estimators=100$, $contamination=0.05$) scores GPS windows across 6 features every 30 seconds via a Celery background worker. Anomaly score < -0.15 triggers automatic Socket.io alert to passenger and admin — no SOS button required.
- **AI-Driven Ride Risk Score:** Pre-trip composite score from driver anomaly rate (40%), inverse rating (30%), time-of-day risk (15%), route zone risk (15%) — displayed as Green/Yellow/Red badge. No commercial platform offers this.
- **Domain-Verified Trust Circles with Reputation Scoring:** Institutional email domain verification auto-assigns communities; in-community reputation score compounds over shared ride history, beyond simple star ratings. Women-only mode is a hard filter with no fallback.
- **Carbon-Aware EV Routing with Gamified Green Rewards:** Per-ride CO₂ savings displayed (Karnataka grid factors: EV 0.05 kg/km, Petrol 0.21 kg/km). Green Points: EV +10, Pool +5, EV+Pool +18; 100 pts = ₹10 fare discount.
- **Unified Safety & Operations Dashboard:** Leaflet.js map with live driver markers (Socket.io, 30s update), anomaly alert feed, community stats, and green impact metrics — the operational nerve centre of the platform.

7. System Architecture

Block Diagram — System Architecture (Phase 1)

The following block diagram illustrates the four-layer architecture of the Smart Ride-Sharing Platform, showing the Client Layer (React.js frontend), Backend Layer (Python Flask with all API blueprints and Celery workers), AI/ML Layer (TF/Keras NLP model, Isolation Forest, spaCy, Risk Score Engine), and Data Layer (MongoDB Atlas and Redis).

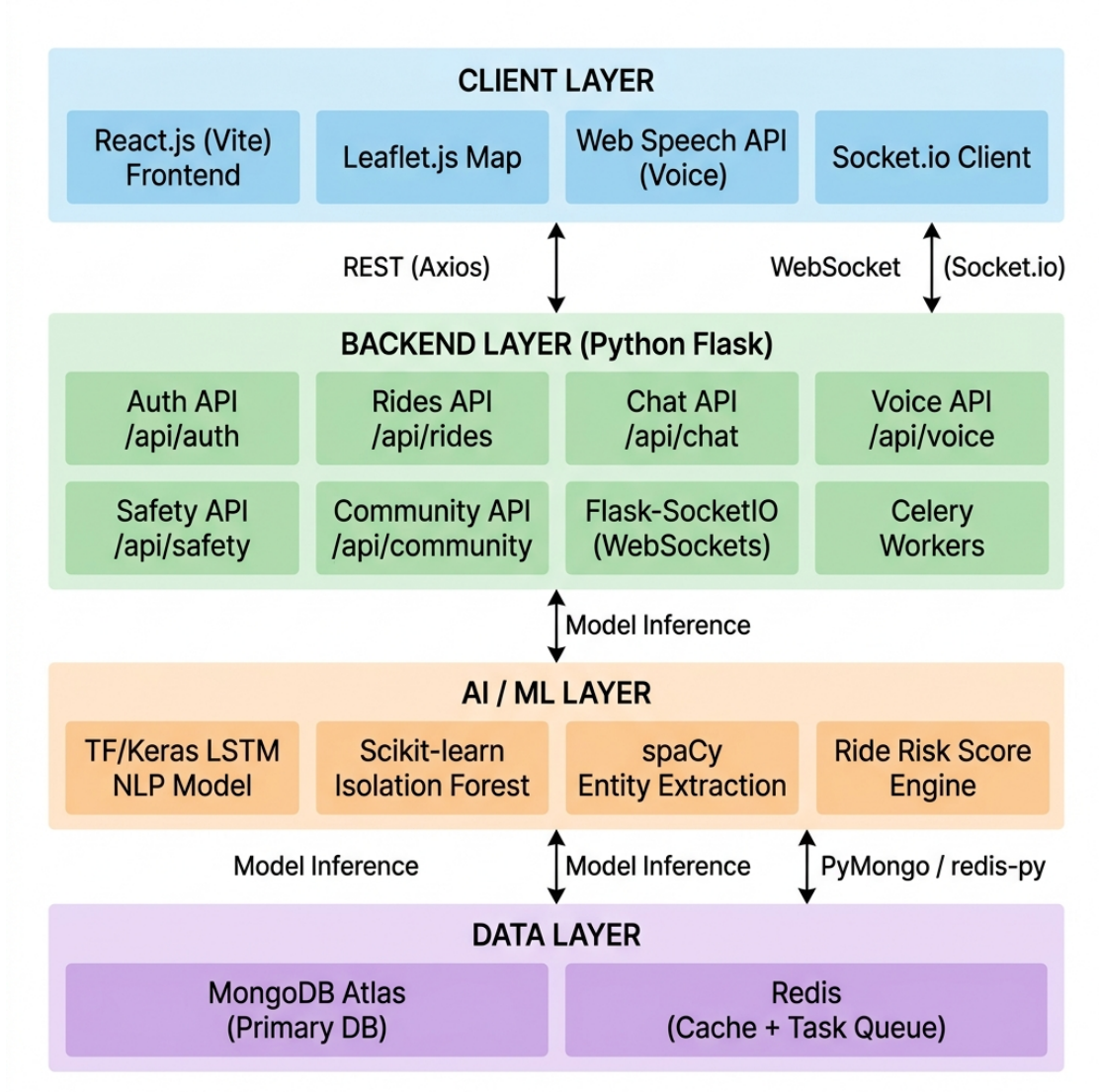


Figure 1: System Architecture Block Diagram — Four-Layer Platform Overview

Data Flow Diagram — DFD (Phase 2)

The Data Flow Diagram illustrates how data moves between the three external entities (Passenger, Driver, Admin) and the six core internal processes: Authentication & Registration, NLP Intent Processing, Ride Booking Service, GPS Anomaly Detection (Silent Operator), Community Verification, and Dashboard & Alerts. Data stores D1–D5 represent the MongoDB collections.

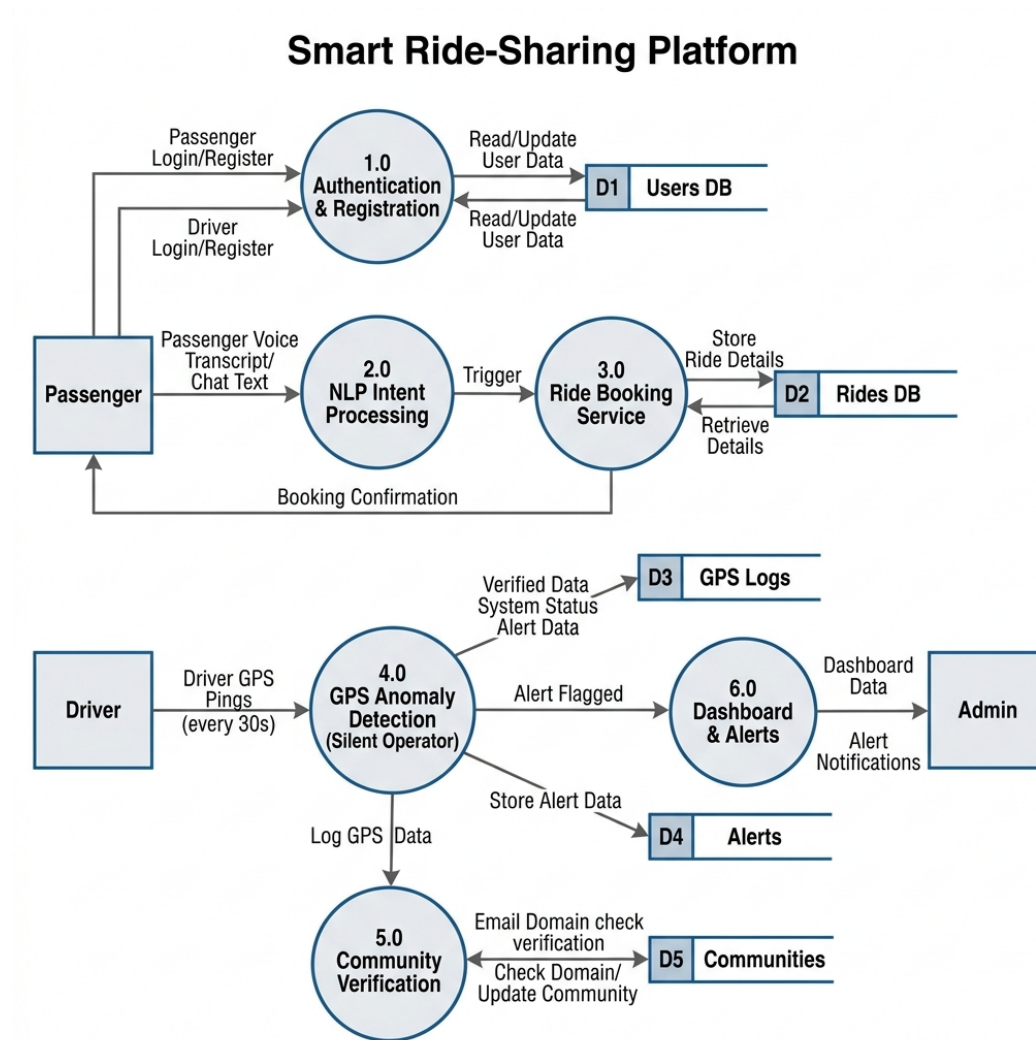


Figure 2: Data Flow Diagram (DFD) — Smart Ride-Sharing Platform

Entity-Relationship Diagram — ER Diagram (Phase 2)

The ER Diagram maps the relationships between the six core database entities: USER, DRIVER, RIDE, COMMUNITY, ALERT, and GPS_LOG. Key relationships include USER books RIDE (1:N), DRIVER accepts RIDE (1:N), RIDE generates ALERT (1:N), RIDE has GPS_LOG (1:N), USER joins COMMUNITY (M:N), and DRIVER belongs to COMMUNITY (M:N).

Smart Ride-Sharing Platform

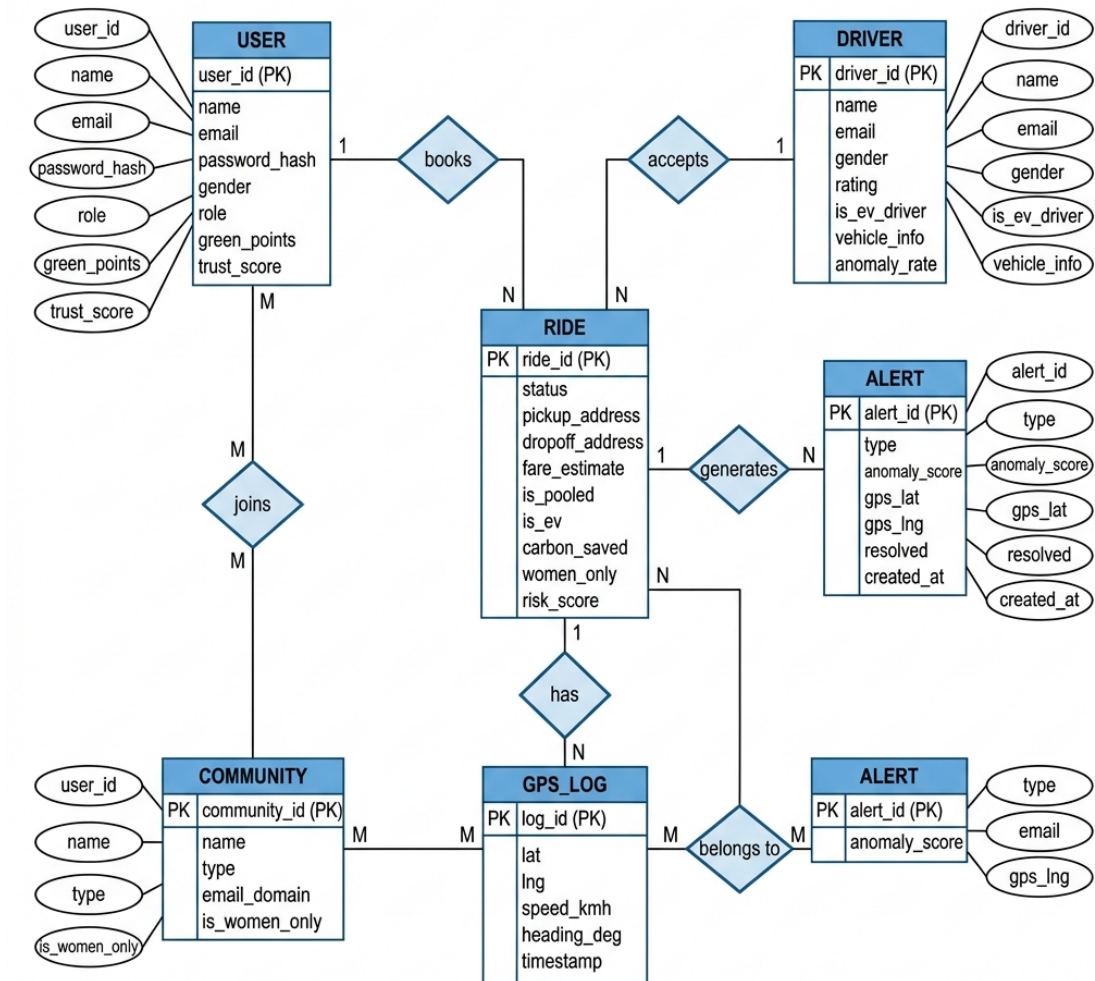


Figure 3: Entity-Relationship (ER) Diagram — MongoDB Collection Schema

System Interaction Flow — Silent Operator Sequence Diagram (Phase 3)

The following UML sequence diagram shows the step-by-step interaction flow of the Silent Operator Safety Monitor — the platform's most technically significant module. It illustrates how a GPS ping from the Driver App flows through the Flask API, Redis cache, and Celery worker, is scored by the Isolation Forest model, and — if anomalous — triggers immediate Socket.io alerts to the Passenger App and Admin Dashboard without any user action.

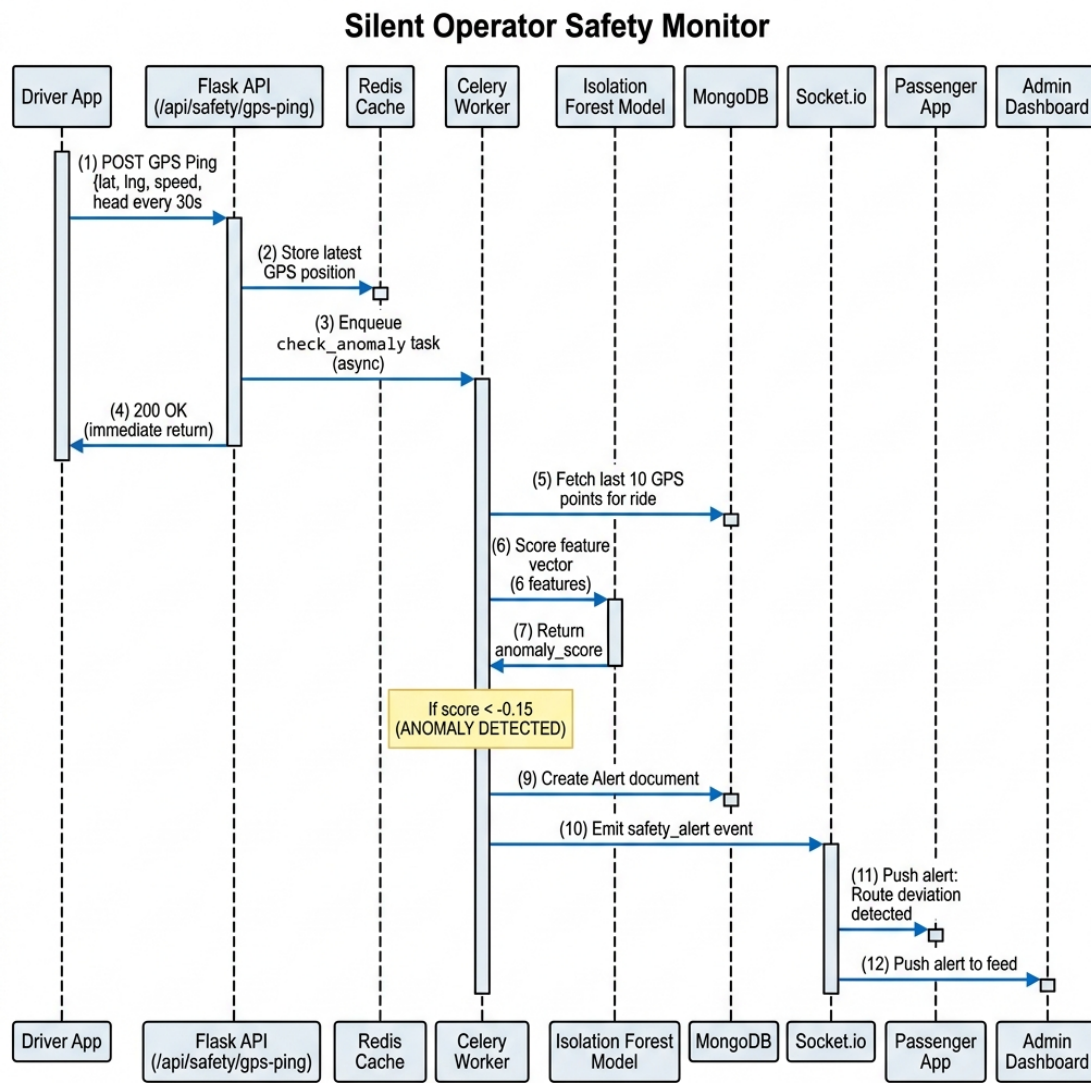


Figure 4: Sequence Diagram — Silent Operator GPS Anomaly Detection Flow

Use Case Diagram (Phase 3)

The Use Case Diagram shows all interactions between the three system actors (Passenger, Driver, Admin) and the platform's use cases. Notable include/extend relationships: Email Domain Verification extends Register; NLP Intent Classification is included in Book a Ride; Anomaly Detection is included in Send GPS Ping.

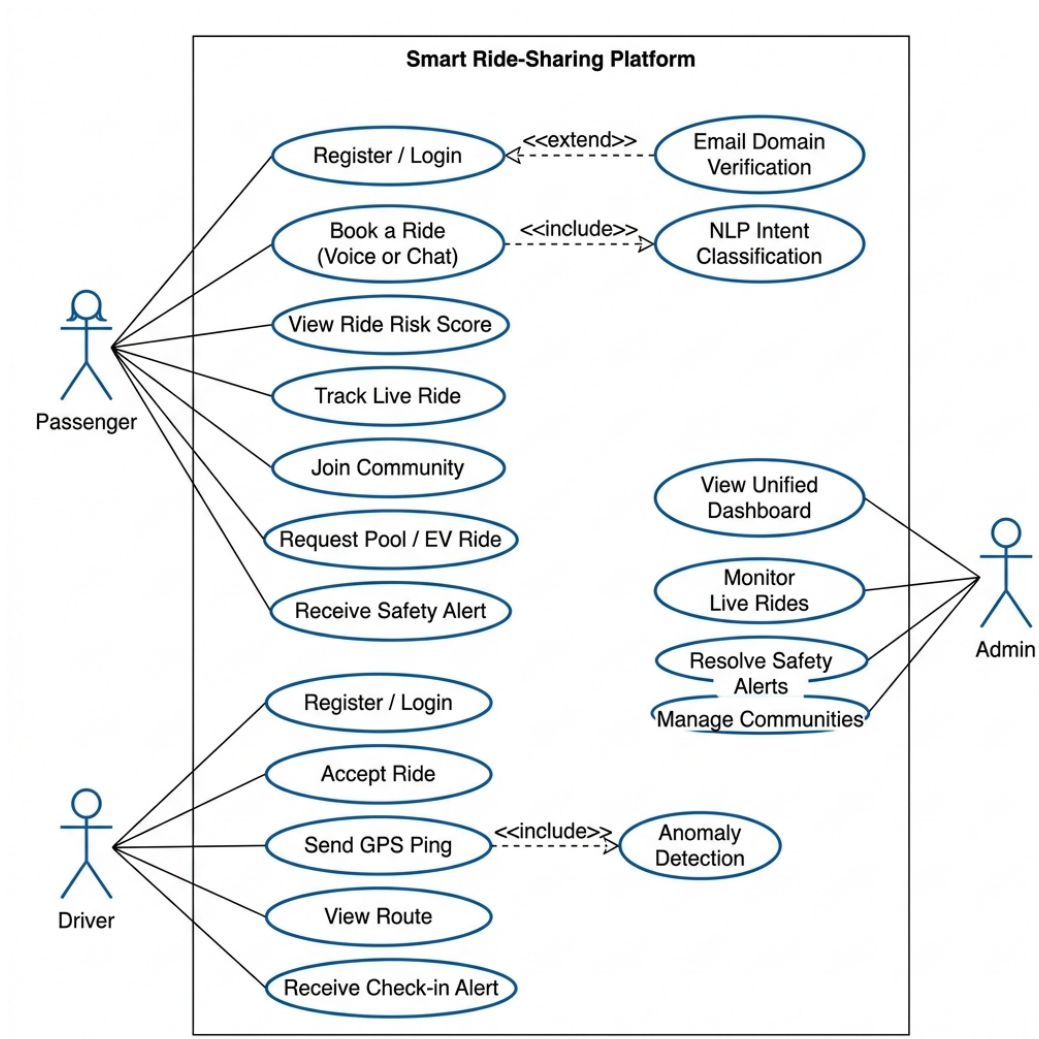


Figure 5: UML Use Case Diagram — Smart Ride-Sharing Platform

8. Modules Description

Module 1: Multi-Turn Intent-Aware NLP Chatbot

A TensorFlow/Keras LSTM neural network classifies user intents from natural language queries across 7 intent classes (booking, cancellations, fare estimates, ride status, pooling, green options, greetings). Unlike rule-based bots, this model is trained on a custom intent dataset (100+ annotated patterns) and maintains session context across multiple conversation turns — enabling follow-up dialogue such as confirming a pickup point after asking "Book a cab." When high-confidence booking or cancellation intent is detected, the chatbot directly triggers the respective backend service API without requiring the user to navigate menus manually.

Module 2: Multilingual Voice Booking (Hindi & English)

The browser-native Web Speech API (configured for 'en-IN' locale, supporting Hindi and English) captures voice commands and converts them to text client-side, sending the transcript to the Flask backend's /api/voice/process endpoint. A spaCy NLP entity extraction pipeline identifies pickup location, dropoff, and preferred ride type. The VoiceButton component provides real-time visual feedback (pulsing animation + live transcript display) during recording.

Module 3: Proactive "Silent Operator" GPS Anomaly Detection Engine

A background AI safety system running entirely without passenger interaction. Scikit-learn's Isolation Forest model (trained on the NYC Taxi Dataset) scores a sliding window of 10 consecutive GPS pings against six features: speed (km/h), heading change (degrees), stop duration (seconds), route deviation (metres via Haversine formula), time-of-day bucket, and distance from the previous point. A Celery background worker invokes this scoring every 30 seconds. If the anomaly decision score falls below -0.15 , an alert document is created in MongoDB and a Socket.io event is immediately pushed to the passenger's device and the admin dashboard — flagging the anomaly type and triggering an automatic driver check-in prompt.

Module 4: AI-Driven Ride Risk Score

Before a passenger accepts a matched ride, the backend computes a composite pre-trip risk score for the assigned driver using four weighted factors: driver's GPS anomaly rate from the past 30 days (40%), inverse star rating (30%), time-of-day risk factor (15%), and route zone risk profile (15%). The score is displayed as a colour-coded badge — Green (Low: 0.0–0.3), Yellow (Moderate: 0.3–0.6), or Red (High: 0.6–1.0) — promoting transparent, informed consent before every trip. No existing commercial platform offers this feature.

Module 5: Domain-Verified Trust Circles with Reputation Scoring

Institutional email domain verification (e.g., @presidency.edu.in → "Presidency College Commuters") automatically places users into verified ride circles on registration. An in-community reputation score is computed from shared ride history, punctuality records, and mutual ratings — creating a trust metric that compounds over time. A hard-filter women-only mode ensures the driver-matching algorithm exclusively assigns female drivers when selected, notifying the passenger of unavailability rather than silently falling back to male drivers.

Module 6: Carbon-Aware EV Routing with Gamified Green Rewards

A ride filter surfaces electric vehicle drivers near the passenger, with per-ride CO₂ savings calculated using Karnataka grid emission factors (EV: 0.05 kg CO₂/km, Petrol: 0.21 kg CO₂/km). For pooled rides, emissions are divided by the number of passengers. The calculated CO₂ offset is displayed on a Green Badge on the ride confirmation screen. A Green Points reward system (EV ride: +10 pts, Pooled: +5 pts, EV+Pooled: +18 pts; 100 pts = ₹10 fare discount) gamifies sustainable travel.

Module 7: Unified Safety & Operations Dashboard

A real-time web interface consolidating all platform activity: a Leaflet.js map with live driver markers updating every 30 seconds via Socket.io, a live anomaly alert feed (flagging unresolved route deviations and unusual stops), active ride count, community membership statistics, and a green impact summary (total CO₂ saved, Green Points distributed). Admin users can resolve alerts and communicate with drivers from this unified view — the operational control centre for the platform.

9. Methodology

The development of the Smart Ride-Sharing Platform follows an Agile Sprint-Based Methodology, structured into clearly defined two-week sprints. This allows iterative development and progressive AI model integration, enabling continuous testing and refinement of the ML components throughout the lifecycle.

Phase 1: Requirement Engineering & Technical Stack Selection

- Tech Stack: React.js (Vite) + Tailwind CSS (frontend), Python Flask with Flask-SocketIO as the sole backend (no Node.js), MongoDB Atlas (NoSQL), Redis (task queue + GPS cache), Celery (background workers).
- Data Acquisition: NYC Taxi Trip Dataset (~1M records) for Isolation Forest training; custom ride-sharing intent dataset (100+ patterns, 7 intents) for the NLP chatbot.

Phase 2: System Architecture & Database Design

- Micro-Modular Design: Seven independent but interlinked modules connected via Flask Blueprints (/api/auth, /api/rides, /api/chat, /api/voice, /api/safety, /api/community, /api/dashboard).
- Data Modeling: ER diagram maps five core MongoDB collections: Users, Rides, Communities, Alerts, GPS_Logs (time-series).
- Security Protocol: JWT-based authentication (Flask-JWT-Extended) with role-based access control; email domain verification automates community assignment.

Phase 3: Iterative Development (Sprints)

- Sprint 1 (Wks 1–2): Project scaffold — Flask app factory, Blueprint stubs, JWT auth API, MongoDB Atlas, Vite + React with login/register pages.
- Sprint 2 (Wks 3–4): Core ride booking CRUD — book, cancel, view rides; MongoDB ride document lifecycle.
- Sprint 3 (Wks 5–6): AI Chatbot and Voice modules — custom dataset creation, Keras LSTM training ($\geq 85\%$ accuracy), /api/chat, ChatWidget; Web Speech API hook, spaCy NER, VoiceButton.
- Sprint 4 (Wks 7–8): Silent Operator and Risk Score — NYC Taxi data prep, Isolation Forest training, GPS ping endpoint, Celery worker, Socket.io alert, pre-trip Risk Score display.
- Sprint 5 (Wks 9–10): Communities and Green Options — email domain verification, reputation scoring, women-only hard filter, pooling algorithm (Haversine), carbon calculator, Green Points system.
- Sprint 6 (Wks 11–12): Unified Dashboard, integration testing, deployment — Leaflet.js map, live driver markers, admin alert feed; Pytest + Jest tests; deploy on Render.com + Vercel.

Phase 4: AI Training & Model Calibration

- NLP Fine-tuning: TF/Keras LSTM (Embedding $\rightarrow$ LSTM(128) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(softmax)), Adam optimiser, EarlyStopping (patience=20). Target: $\geq 85\%$ validation accuracy.
- Safety Calibration: Isolation Forest ($n_estimators=100$, $contamination=0.05$) threshold (-0.15) calibrated by injecting synthetic anomalies (route deviations, unusual stops) into test sequences to minimise false positives.

Phase 5: Quality Assurance & Integration Testing

- Integration Testing: Voice Module transcript pipeline correctly triggers /api/rides/book when ride intent and entities are extracted.
- Edge-Case Simulation: Women-only hard filter tested to confirm no male drivers are returned under any condition.
- Anomaly Injection Testing: Silent Operator correctly fires Socket.io alerts when synthetic anomalous GPS sequences are injected.
- UAT (User Acceptance Testing): Unified Dashboard real-time map and alert feed validated via WebSockets with simulated concurrent rides.

Phase 6: Deployment & Documentation

- Flask backend deployed on Render.com (Gunicorn + Eventlet worker); React frontend on Vercel; MongoDB Atlas M0 free cluster.

10. Tools and Technologies

Hardware

Component	Specification
Processor	Intel i5/i7 or equivalent
RAM	8GB+
Storage	256GB SSD

Software

Layer	Technology
Client Side	HTML5, CSS3, JavaScript, React.js (Vite), Tailwind CSS, Leaflet.js
Server Side	Python Flask, Flask-SocketIO, Flask-JWT-Extended, Celery
AI / ML	TensorFlow/Keras (NLP chatbot intent classifier), Scikit-learn Isolation Forest (anomaly detection), spaCy (NER), Web Speech API (voice)
Database	MongoDB Atlas (NoSQL — users, rides, communities, alerts, GPS logs)
Cache / Queue	Redis (Celery task broker + GPS position cache)
App Server	Render.com (backend), Vercel (frontend)
IDE	VS Code
Version Control	Git + GitHub
Testing	Pytest (backend), Jest + React Testing Library (frontend)

13. Expected Outcomes

The Smart Ride-Sharing Platform will deliver measurable improvements in user experience, safety, and efficiency for urban transportation.

- Achieve $\geq 85\%$ validation accuracy in the TensorFlow/Keras NLP chatbot intent classifier, reducing manual support dependency by 70% and compressing query response time from minutes to under 2 seconds.
- Demonstrate proactive ride safety via the Silent Operator anomaly detection engine — automatically flagging route deviations, unusual stops, and speed violations without any user action. No commercial platform currently provides this.
- Introduce the AI-Driven Ride Risk Score — a pre-trip passenger safety transparency feature (driver GPS anomaly history, ratings, time-of-day, route zone risk) — the first such feature in the ride-sharing domain.
- Increase user trust and retention in safety-sensitive groups through domain-verified micro-communities and hard-filter women-only mode, targeting 50% higher engagement from female commuters and institutionally affiliated users.
- Facilitate 50% estimated cost reduction for intelligent ride pooling users while quantifying and displaying per-trip CO₂ savings and incentivising eco-friendly choices through the Gamified Green Rewards (Green Points) system.
- Demonstrate a fully integrated, scalable ML-in-production prototype combining NLP, multilingual speech recognition, Scikit-learn anomaly detection, and real-time WebSocket communication.

14. Future Enhancements

- Integrate LSTM-based demand prediction to forecast peak-hour ride demand by zone and enable proactive driver allocation, reducing average passenger wait times.
- Implement dynamic pricing algorithms using Reinforcement Learning for fair, real-time surge price adjustments that balance driver incentives with passenger affordability.
- Expand multilingual NLP support to Kannada, Tamil, and Telugu to widen accessibility across South Indian urban markets.
- Develop sentiment analysis on chatbot interaction logs to continuously fine-tune the NLP intent model based on real-world user language patterns and satisfaction signals.
- Add augmented reality (AR) ride tracking with live 3D maps and driver-passenger video check-ins for enhanced situational awareness.
- Incorporate blockchain-based ID verification for tamper-proof community membership records and immutable ride history.
- Expand to multi-modal transport integration (city buses, metro connections) with AI route optimisation across vehicle types for complete urban mobility coverage.
- Introduce federated learning for the Silent Operator model — improving anomaly detection from GPS data across all rides without centralising raw location data, enhancing both accuracy and user privacy.

Approval

Guide Name & Signature: _____

Project Coordinator: _____

Head of Department: Dr. Alli.A